

Exercise1_solved_mysolutions

November 18, 2024

Q1: Factorial of Number through Recursion

```
[164]: def factorial(n:int) -> int:
        if n == 0:
            return 1
        return n * factorial(n-1)

factorial(5)
```

[164]: 120

Q2: Common elements from a list of numbers

```
[165]: def find_intersection_bruteforce(list_1: list, list_2: list) -> list:
        """O(n^2) algorithm"""
        list_3 = []
        for list_item in list_1:
            if list_item in list_2:
                list_3.append(list_item)

        return list_3

def find_intersection_set(list_1: list, list_2: list) -> list:
    """Inbuilt set algorithm"""
    return set(list_1).intersection(list_2)

# Faster way - cast to sets, and use intersect.

list_1 = ["Hello", "Buffalo", "One"]
list_2 = ["Hello", "two", "Buffalo"]

# find_intersection_bruteforce(list_1, list_2)
find_intersection_set(list_1, list_2)
```

[165]: {'Buffalo', 'Hello'}

Q3: Frequency of each element in a list

```
[166]: example_input = [1,2,2,3,3,3,4]

from collections import Counter

def frequency_map (given_list: list) -> dict:
    frequency_map = {}

    for element in given_list:
        frequency_map[element] = frequency_map.get(element, 0)+ 1

    return frequency_map

def frequency_map_with_counter(given_list: list) -> dict:
    return dict(Counter(given_list))

print(frequency_map(example_input))
print(frequency_map_with_counter(example_input))
```

```
{1: 1, 2: 2, 3: 3, 4: 1}
```

```
{1: 1, 2: 2, 3: 3, 4: 1}
```

Q4: De duplication while preserving order

```
[167]: example_input = [1,1,2,3,4,3]

def deduplicate(given_list: list) -> dict:
    return list(dict.fromkeys(given_list))

print(deduplicate(example_input))
```

```
[1, 2, 3, 4]
```

Q5: Binomial Coefficient

```
[168]: # Example Input
n = 5
r = 3

def binomial_coeff(n: int, r: int) -> int:
    return factorial(n) / (factorial(r) * factorial(n-r))

binomial_coeff(n,r)
```

```
[168]: 10.0
```

Q6. String Reversal

```
[169]: # reference_input = "hello"
```

```
def reverse(given_string: str):
    max_index = len(given_string) - 1
    return "".join(given_string[max_index - i] for i in range(max_index+1))

def driver_q6():
    # given_string = input("Enter a string: ")
    given_string = "Hello"
    print("Given string: {}".format(given_string))
    print("Reversed string: {}".format(reverse(given_string)))

driver_q6()
```

Given string: Hello
Reversed string: olleH

Q7. Leap Year Check

```
[170]: def check_leap_year(given_year: int) -> bool:
        is_multiple_of_4: bool = given_year % 4 == 0
        is_multiple_of_100: bool = given_year % 100 == 0
        is_multiple_of_400: bool = given_year % 400 == 0

        return is_multiple_of_400 or (is_multiple_of_4 and not is_multiple_of_100)

def driver_q7():
    given_year = int(input("Enter year to check: "))
    print(check_leap_year(given_year))

#driver_q7() ## Uncomment to use this question
```

Q8. Count No. of Vowels in a string

```
[171]: from collections import Counter

vowel_list = ['a', 'e', 'i', 'o', 'u']

def count_vowels(given_str: str) -> int:
    char_freq_map = Counter(given_str)
    return sum([char_freq_map.get(vowel, 0) for vowel in vowel_list])

given_str = "hello world"
count_vowels(given_str)
```

[171]: 3

speeding up from this point on to finish the assignment

Q9. Min-Max tuple fro int list

```
[172]: given_input = [1,2,3,4,5]
result_tuple = (max(given_input), min(given_input))
print(result_tuple)
```

(5, 1)

```
[173]: #Q10. Sum of all even numbers in list
given_input = [1,2,3,4,5]
result = sum([i for i in given_input if i%2 == 0])
print("Q10 answer:", result)
```

Q10 answer: 6

```
[174]: # Q11. Second largest

def get_second_max_element(given_input: list) -> int|None:
    shrunk_list = [element for element in given_input if element !=
    ↪max(given_input)]
    return max(shrunk_list) if shrunk_list else None

given_input_1 = [1,2,3,4,5]
given_input_2 = [2,2,2,2,2]

print("Q11 a) {} \nQ11 b) {}".
    ↪format(get_second_max_element(given_input_1),get_second_max_element(given_input_2)))
```

Q11 a) 4

Q11 b) None

```
[175]: # Q12. Remove all whitespace characters
def strip_all_whitespace(given_string: str) -> str:
    return "".join(given_string.split())

example_input = "hello world"
print(strip_all_whitespace(example_input))
```

helloworld

```
[176]: # Q13. Length of longest word in string
def len_of_longest_word(given_string: str) -> str:
    words = given_string.split()
    return len(max(words))

example_input = "The quick brown fox"
print(len_of_longest_word(example_input))
```

5

[177]: # Q14. Multiplication table for given number from 1 to 10:

```
def get_table_key(given_number, index):
    return "{} x {}".format(given_number, index)

def get_table_value(given_number, index):
    return given_number * index

def get_table_entry(given_number, index):
    return "{} = {}".format(
        get_table_key(given_number, index), get_table_value(given_number, index)
    )

def get_multiplication_table(given_number: int, table_start_pos, table_end_pos) → list:

    multiplication_table = [
        get_table_entry(given_number, index)
        for index in range(table_start_pos, table_end_pos + 1)
    ]
    return multiplication_table

def print_multiplication_table(given_number: int) → None:
    table_start_pos = 1
    table_end_pos = 10
    multiplication_table = get_multiplication_table(given_number,
    →table_start_pos, table_end_pos)
    print("\n".join([element for element in multiplication_table]))

example_input = 3
print_multiplication_table(example_input)
```

```
3 x 1 = 3
3 x 2 = 6
3 x 3 = 9
3 x 4 = 12
3 x 5 = 15
3 x 6 = 18
3 x 7 = 21
3 x 8 = 24
3 x 9 = 27
3 x 10 = 30
```

[179]: *# Q15. Print all prime numbers in a range*

```
def is_prime(given_number: int) -> bool:
    if n == 1:
        return False

    i = 2

    while i*i <= given_number:
        if (given_number % i == 0):
            return False

        i += 1

    return True

def print_all_primes_in_range(start_pos, end_pos):
    primes = [number for number in range(start_pos, end_pos+1) if
↪is_prime(number)]
    print(primes)

example_input = (10, 20)
print_all_primes_in_range(*example_input)
```

[11, 13, 17, 19]